In-Class Activity 4: Sequence and State Machine Diagrams

Artie Bowman · CIS 641 · October 11, 2026

Task

The activity asks for a **sequence diagram** and a **state machine diagram** for the term project. Both are behavioral models from Chapter 6. A sequence diagram shows the order of messages between objects for one use case. A state machine shows the stages one object goes through in its life and what moves it from one stage to the next.

Project Recap

My term project is PersonaSync™. In one sentence: the user writes down who they are and what they are working on once, and PersonaSync carries that context into whatever AI tool they open, so they stop re-explaining themselves. The part I am building this semester is the delivery piece, the client that puts the context into the AI tool. One rule runs through the whole product: the user writes and confirms everything the system knows about them. Anything an AI guesses is tagged as a suggestion and stays out until the user approves it. Both diagrams show that rule in action.

What Is Attached

The class names match the Week 5 CRC cards and class diagram, so the three activities line up. This document has four parts:

- **Section 1:** the sequence diagram for the Deliver Context use case, with a short walkthrough (Figure 1).
- **Section 2:** the state machine for a ContextBlock, the object that carries the confirmation rule (Figure 2).
- **Figures:** the two diagrams on their own pages, plus Figure 3, the rough sequence diagram I drew by hand in class for a second use case, Create Persona.
- **Appendix:** a template and rules for each diagram type. I am keeping it so the next diagrams for the SRS are faster to make and look the same as these.

1. Sequence Diagram: Deliver Context into an AI Session

What it shows

This is the main thing the project does. The user opens an AI tool. The client app notices and offers the persona they usually use there. The user picks one. The app then collects that persona's context, shapes it to fit the tool, and pastes it in. At the end it writes a log entry, so the user can always see what was shared and where.

How to read it

The boxes across the top are the parts of the system. Time runs down the page. A solid arrow is one part asking another to do something. A dashed arrow is the answer coming back. The shaded bar under a box means that part is busy. The box labeled *opt* is a step that only happens when the text is too long for the tool. The log entry box sits lower than the others because it does not exist until that step creates it.

Where the confirmation rule shows up

Step 7 only pulls context the user has confirmed, so nothing an AI guessed goes out. Step 12 writes the log as part of the delivery, so there is never a delivery without a record.

> **Example:** I open Claude. The client app offers my “Grad School” persona because that is the one I usually use there. I confirm. The app pastes my context into Claude, and the log shows Grad School went to Claude at 6:05 PM.

See attached diagram, Figure 1 ↓

2. Behavioral State Machine: ContextBlock

What it shows

A ContextBlock is one fact the user keeps about themselves, like “working on a master's in AI at GVSU.” This diagram follows one fact through its life. A fact gets in one of two ways. The user types it, or an AI or a connected tool suggests it. Either way, the user has to confirm it before it counts. Only confirmed facts ever get sent to an AI tool. The states are:

- **Draft:** the user is typing or editing it.
- **Proposed:** an AI or a connected tool suggested it. The user can see it, but it is not sent anywhere until they keep it.
- **Kept:** the user confirmed it. This is the only state that can be sent to a tool.
- **Shared:** it has been sent at least once, and the log says where. Taking back the last share moves it back to Kept.
- **Archived:** hidden from view but still on record. It can come back, or be deleted after one more confirmation.

How to read it

The filled dot is where a fact starts. The bullseye is where it ends. Each arrow is labeled with what causes the move. Anything in square brackets is a condition, and in this diagram every condition is the user confirming something.

> **Example:** Wispr Flow, a voice note tool, suggests “working on CIS 641 term project” from a meeting. It sits in Proposed until I keep it. Once I keep it, it can go to Claude and becomes Shared. If I take that share back it returns to Kept, not Proposed, because I already confirmed it.

See attached diagram, Figure 2 ↓

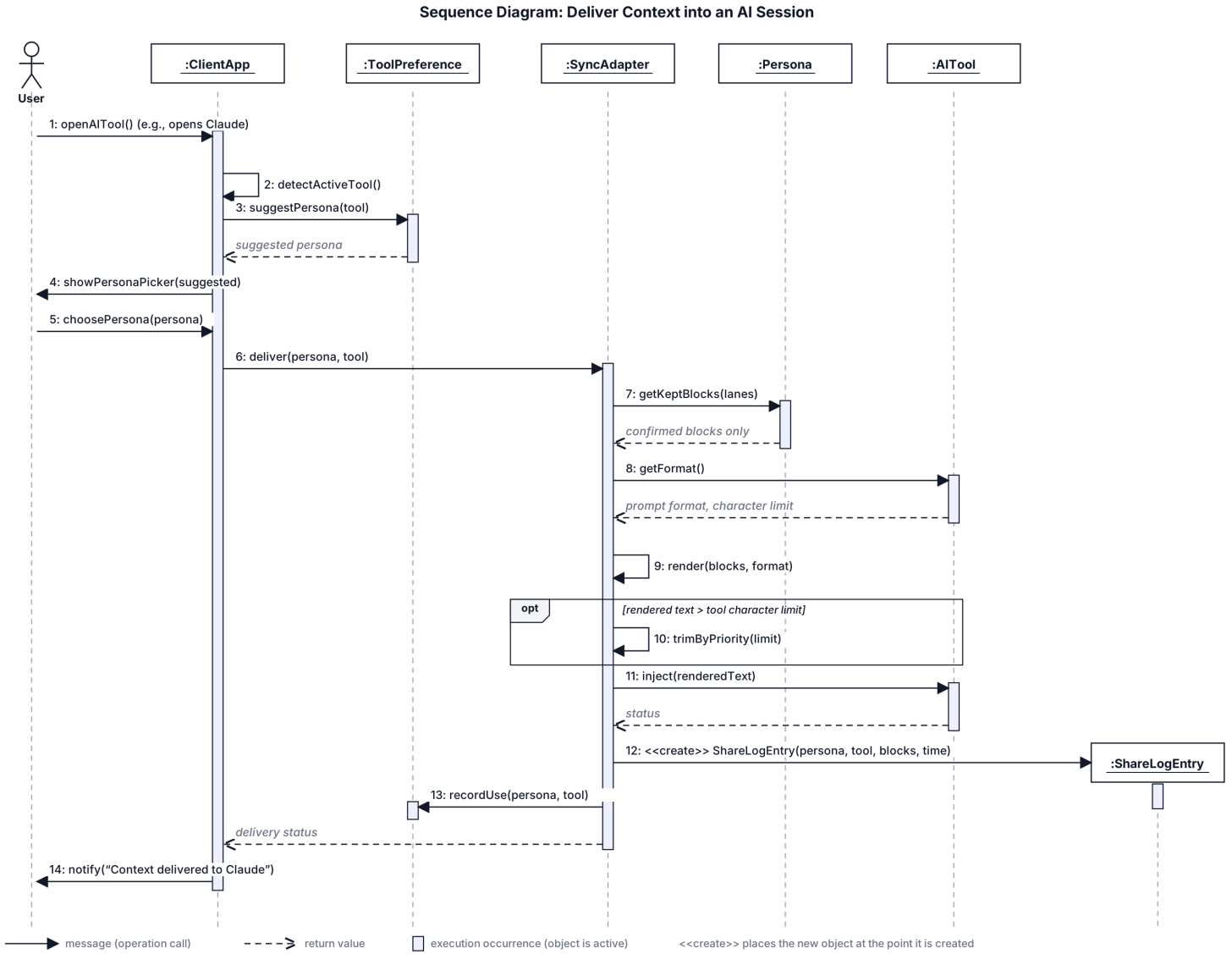


Figure 1. Sequence diagram for the Deliver Context use case. Solid arrows are messages, dashed arrows are return values, and the shaded bars show when each object is active. The ShareLogEntry is drawn where it is created.

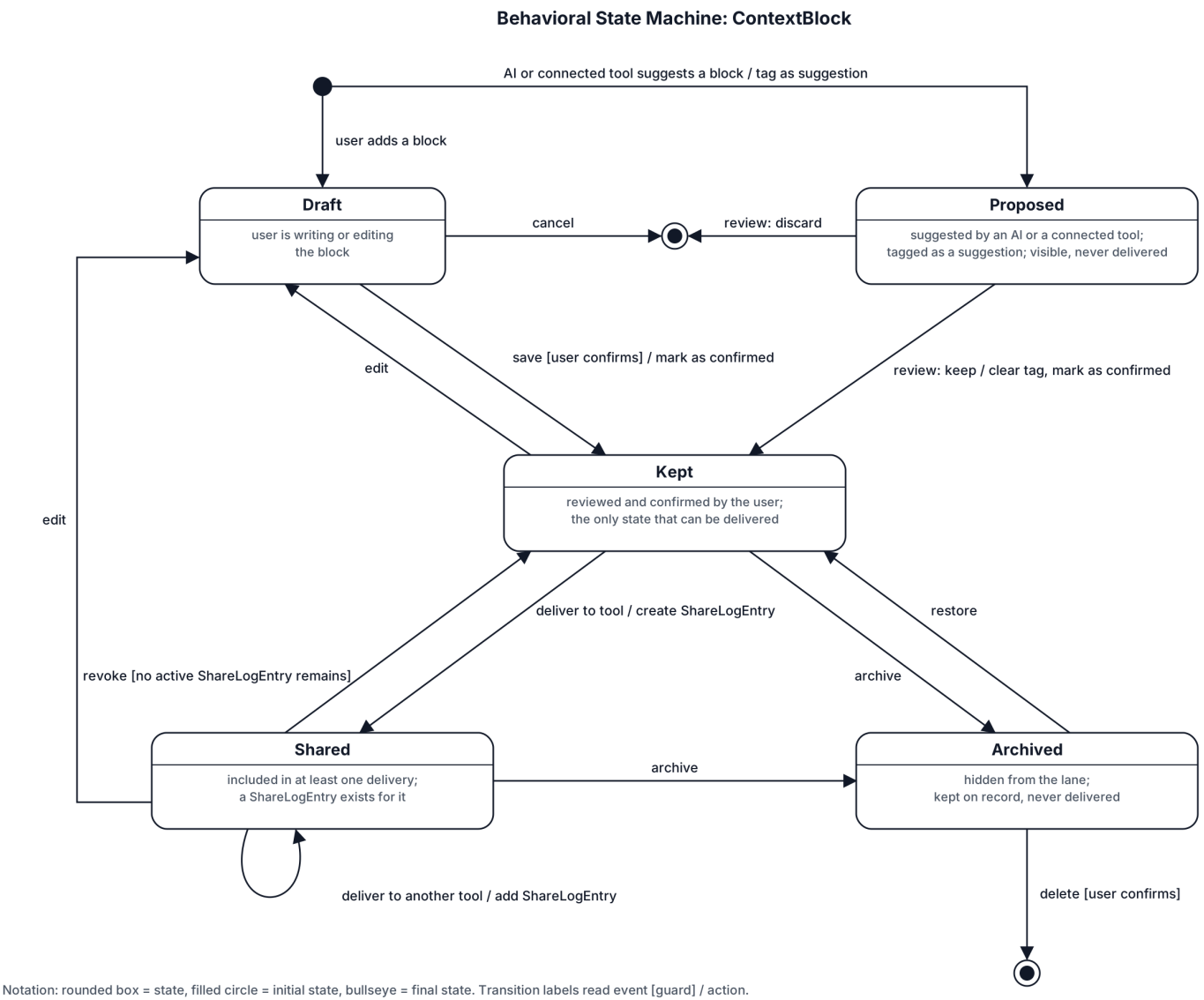


Figure 2. Behavioral state machine for ContextBlock. Kept is the only state that can be delivered. Both entry paths pass through a user confirmation before reaching it.

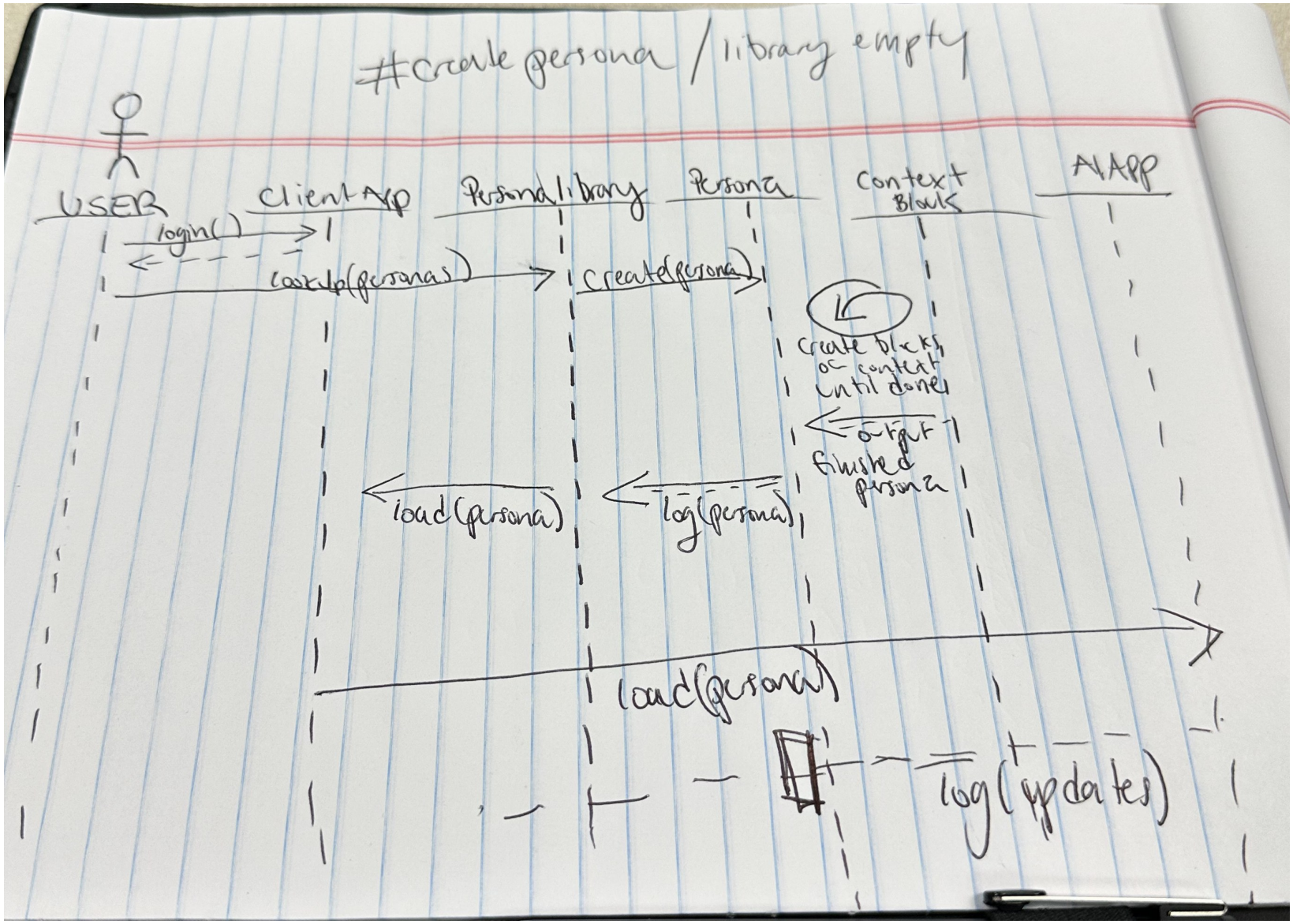


Figure 3. Hand-drawn sequence diagram from class for the Create Persona use case (persona library empty). Rough first pass; the activity-period sketch is a draft, so a few labels are shorthand.

Appendix: Templates and Rules

This is the recipe I used for the two diagrams, kept here so I can reuse it for the next ones. It follows the appendix in In-Class Activity 3.

A.1 Sequence diagram template

One use case per diagram. The pieces and what they mean:

Stick figure	An actor. The person or outside system that starts the use case. Far left.
Box, underlined :ClassName	An object. One per class that takes part, taken from the class diagram.
Dashed vertical line	Lifeline. The object exists for as long as the line runs.
Thin shaded bar	Execution occurrence. The object is busy during that span.
Solid arrow, filled head	Message. One object asking another to do something. Numbered, named like an operation: verb().
Dashed arrow, open head	Return. The answer coming back. Only drawn when the value matters.
<<create>>	The message that makes a new object. Its box is drawn at that height, not at the top.
Box labeled opt, alt, loop	A fragment. opt runs once if its [condition] is true, alt picks one branch, loop repeats.

Rules I follow: Time runs down. Put the actor first and the objects in the order they get called, so arrows stay short. Name each message like an operation from the class diagram. A part that calls itself gets an arrow that loops back to its own line. Past about fifteen messages, split the use case.

A.2 Behavioral state machine template

One object per diagram. The pieces and what they mean:

Filled dot	Initial state. Where the object's life starts. One per diagram.
Rounded box	A state. Something true of the object for a while. Named with an adjective or past participle: Draft, Kept, Archived.
Arrow with label	Transition. The label reads event [guard] / action: the event causes the move, the guard must be true, the action happens on the way.
Bullseye	Final state. The object is done. There can be more than one.
Arrow back to its own box	Self-transition. The event happens and the object stays where it is.

Rules I follow: Every state needs a way in and a way out, except the end states. Put the normal path top to bottom and the exceptions off to the sides. A condition in brackets is where a person decides something. Draw a two-way move as two parallel lines, not one line with two heads. Keep lines straight. A curve usually means two boxes are on the wrong side of each other.

A.3 How the files were made

Both diagrams are drawn as SVG from Python scripts, so every box and arrow has a set position and edits are text edits. The PDF is built with ReportLab. The sketch is a photo cropped to the notebook page. The scripts and images live in the course repo next to the SRS.